

Constrained Horn Clauses and their Applications

Lecture 4

Orna Grumberg, Technion

Condition Synthesis Realizability via Constrained Horn Clauses

Bat-chen Rothenberg, Orna Grumberg, Yakir Vizel, Eytan
Singher

NFM 2023

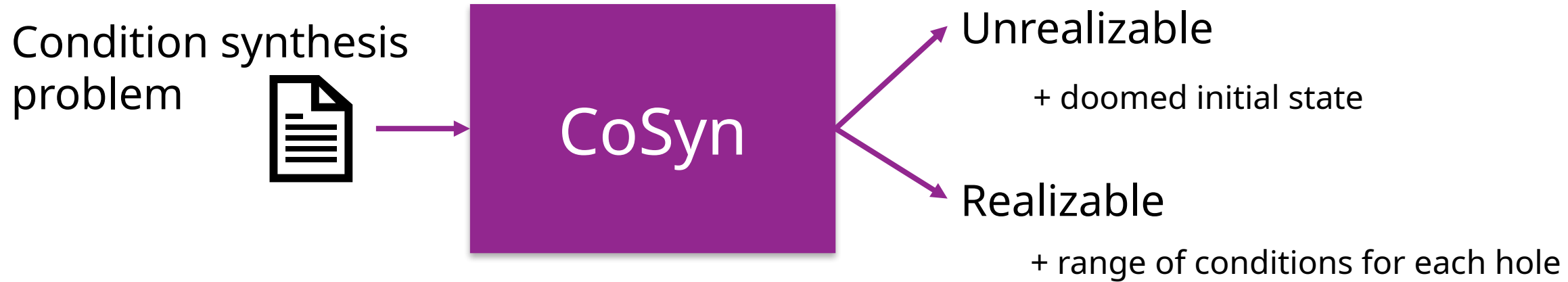
Condition Synthesis

- Receives as input a **partial program**, where conditions are missing in conditional branches (e.g., if/while statements), and a **specification**
 - As **assertions** in the code
- A **solution** to this problem is a set of conditions to fill-in the holes such that the resulting program **meets the specification**.
- **Motivation:** automated program repair

Doomed initial states

- An initial state is called *doomed* if it eventually reaches an error state along *every* run of the program.
- For a doomed initial state σ , there is no way to make the program safe by forcing σ (via conditions) to follow one computation or another. *It will lead to a failure anyway.*
- The problem is **unrealizable** iff there exists a **doomed initial state**

Correctness Evidence



Reduction to CHC

- The problem is **unrealizable** iff there exists a **doomed initial state**.
- To check for the existence of a doomed initial state, COSYN uses a **reduction** to **Constrained Horn Clauses (CHC)**.

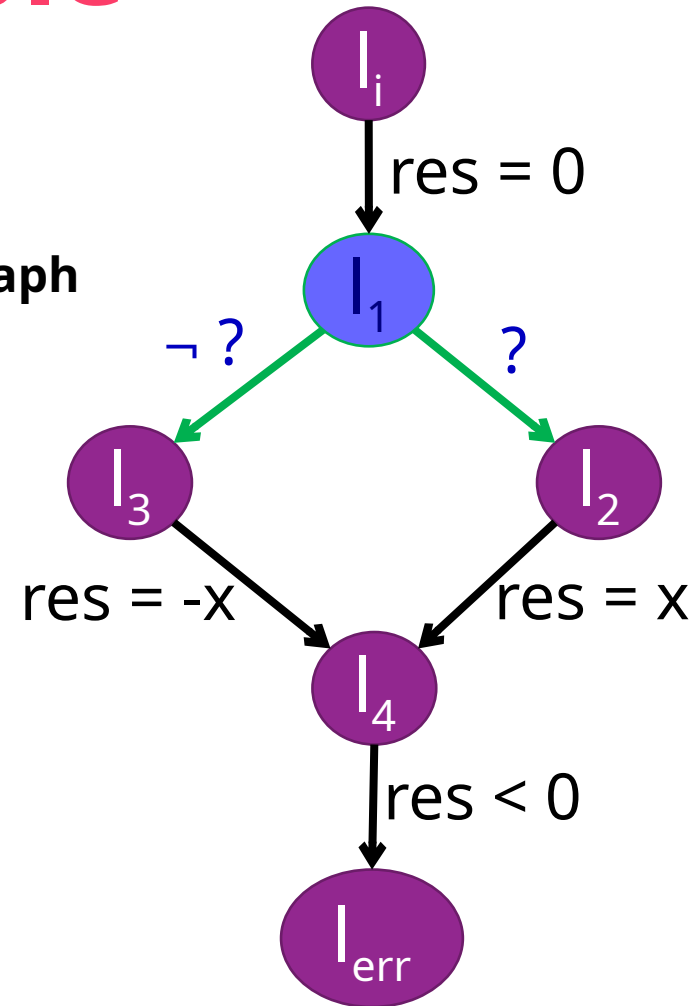
Reduction to CHC – cont.

- A state is doomed at location l if along every path from l to the assertion, the assertion fails.
- Our reduction calculates doomed states **backwards**, starting from the **negation of the assertion**.

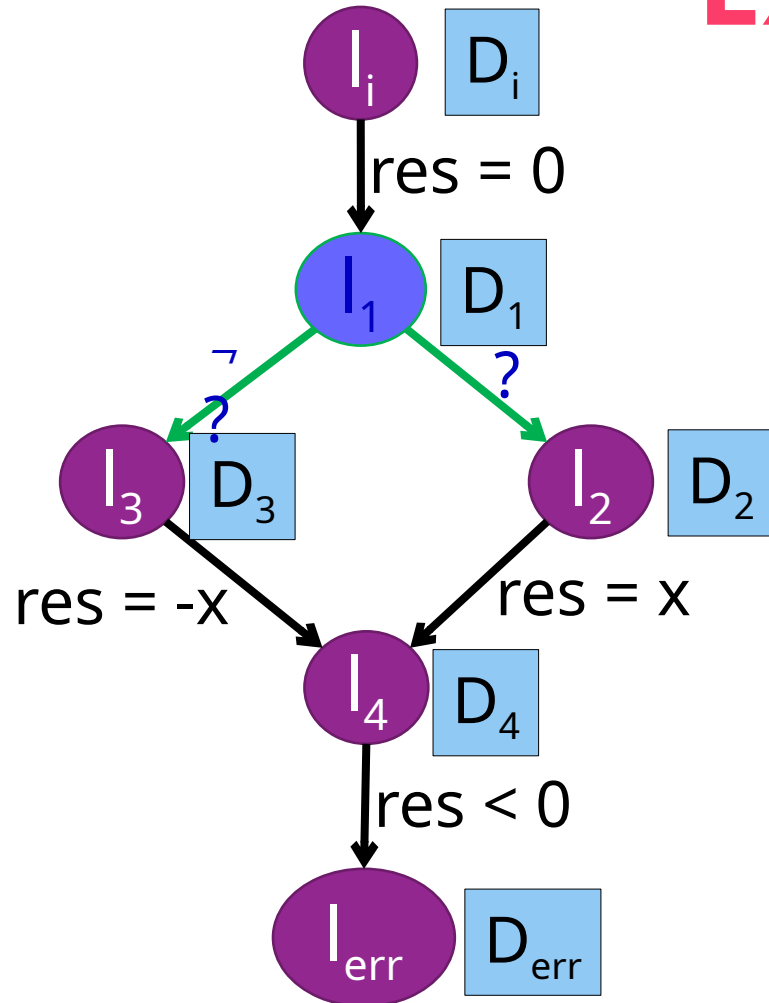
Condition Synthesis Running Example

```
abs(x):  
  res = 0  
  if (?)  
    res = x  
  else  
    res = -x  
  assert (res >= 0)
```

Control-Flow-Graph
(CFG)



Condition Synthesis Running Example



I

D



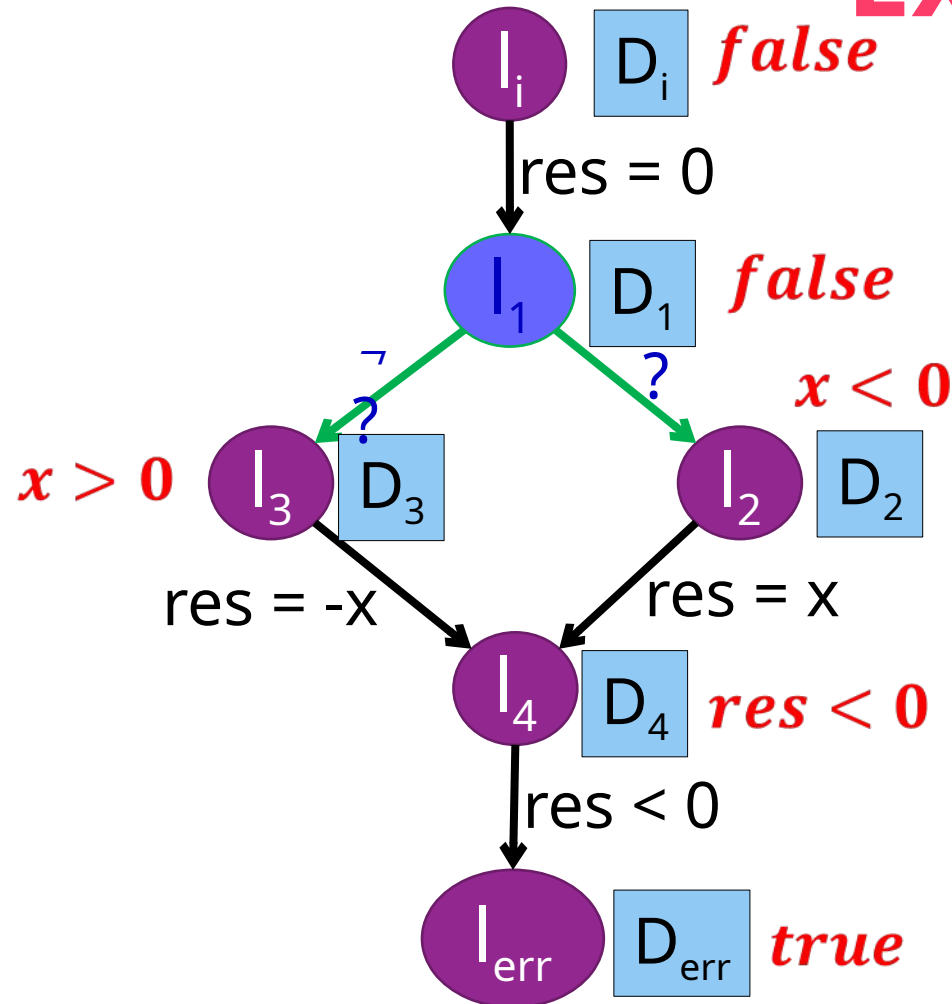
D

D

$$D_{err}(x, res) \wedge (res < 0) \rightarrow D_4(x, res)$$

$$true \rightarrow D_{err}(x, res)$$

Condition Synthesis Running Example



L

D

D

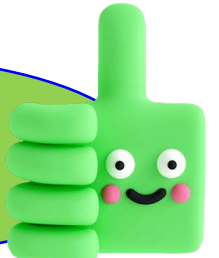
D

D

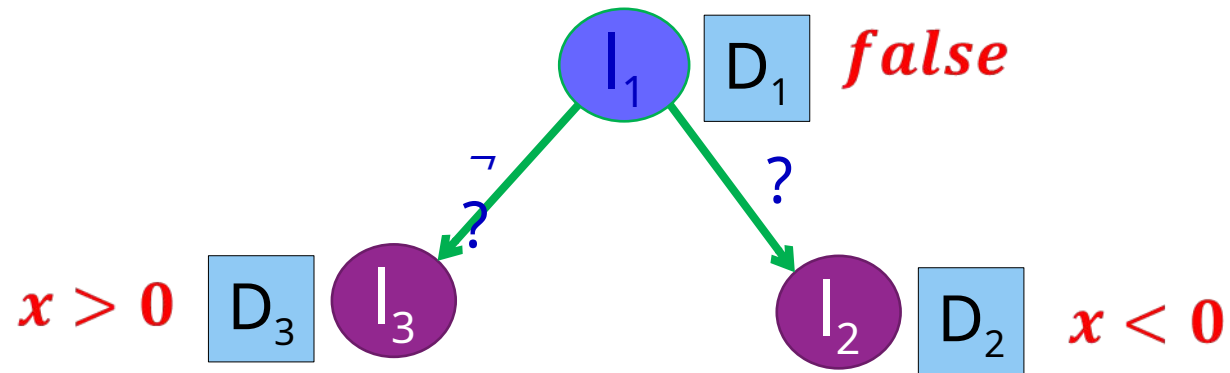
$$D_{err}(x, res) \wedge (res < 0) \rightarrow D_4(x, res)$$

$$true \rightarrow D_{err}(x, res)$$

CHC is
realizable!

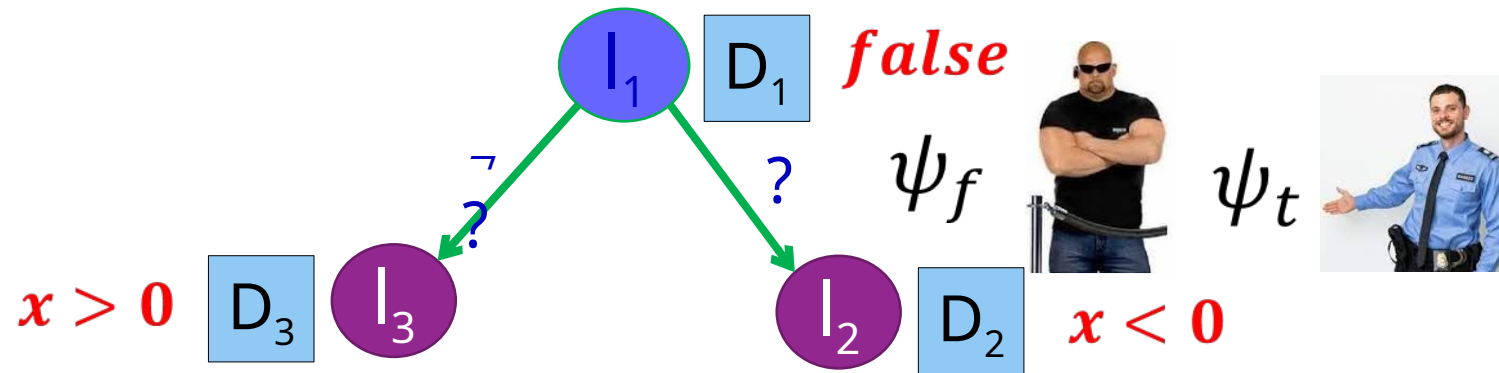


Condition Synthesis Running Example



How to find the range of conditions for L1?
Freedom is obtained from non-reachable states
at I_1 and
from states that are not doomed at either I_2 or I_3

Condition Synthesis Running Example



Every predicate ψ s.t.

$$\psi_f \rightarrow \psi \rightarrow \psi_t$$

is a

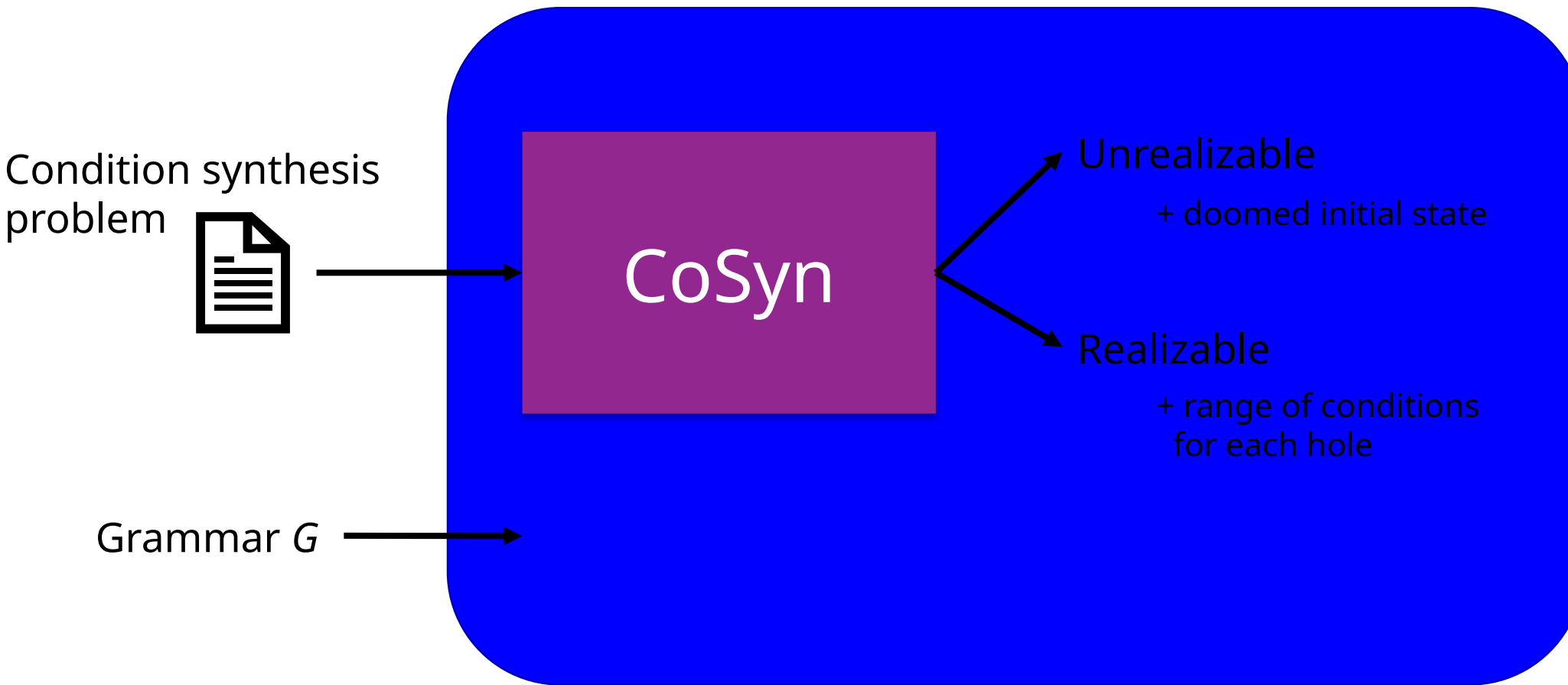
solution

Note: ψ_f, ψ, ψ_t are all conditions that send control to I_2

Combination with SyGuS

- SyGuS is a prominent framework for program synthesis with respect to a formal specification. It limits the search-space to a user-defined grammar G .
- While CoSyn in itself cannot provide a solution within a specific grammar, it can be combined with a SyGuS solver to enhance its performance.

Combination with SyGuS



Combination with SyGuS

It is possible to split the synthesis of different holes into different SyGuS queries!

Condition synthesis problem



CoSyn

Unrealizable

Realizable

+ range of conditions for each hole

SyGuS

Unrealizable

Grammar G

Unrealizable

Realizable

+ solution in grammar



Conclusion

- We present a novel approach for the (un)realizability of the condition synthesis problem
 - via the existence of initial doomed states
 - via reduction to CHCs
- CoSyn accompanies its results with **evidences**
- We show that combining CoSyn with SyGus results in
 - **Definite** result for the **unrealizable** cases
 - **More syntactically appealing** conditions for the **realizable** cases

Questions?

Application 2 of CHCs

CTL* verification and Synthesis using Existential Constrained Horn Clauses (EHC)

Mishel Carelli and Orna Grumberg
ATVA 2024

Setting:

Infinite-state programs with FO-CTL* specification

- e.g. $AF(a \leq b)$, $EG(a = b)$

Goal:

- **Verification** of a program with respect to CTL*
- **Synthesis** of a partial program to a full program which satisfy the CTL* specification

Difference between the two applications

- CTL* specification as opposed to assertions
- In partial programs, conditions and assertions may be missing
- Needs to use Existential Horn Clauses rather than the standard CHC
- Computing is done forward, not backward

Limitation of Horn clauses

- CHCs are **not sufficient** to check **liveness** and **existential properties** (e.g. **AFp**, **EGp**).
- **Existentially quantified Horn Clauses (EHC)** are a special type of Horn-like clauses that allow
 - **existential quantification** and
 - **well-foundedness** (nonexistence of infinite decreasing chain)

In this work, we reduce the problems of CTL* verification and partial synthesis to the problem of EHC satisfiability.

Summary

- We developed a **translation** from the CTL* verification problem to the problem of EHC satisfiability
- We developed a **translation** from CTL* partial synthesis problem to the problem of EHC satisfiability
- Both translations are **sound** and **complete**

- Not implemented
 - The EHC solver is not active anymore
- Possible alternative for future work:
pfwCSP by Unno, Terauchi, Koskinen
 - The authors conjectured that pfwCSP and EHC are inter-reducible

Summary

In this series of talks we saw

- A **theory modular** approach to solving CHCs
 - Based on CHC solvers as a blackbox
- A non-standard use of CHCs for **synthesis**
- A **variation of CHCs** (EHC) that can do much more than standard CHCs
 - BUT – need a working solver

Thank you!